

SMARTTRAVEL MCP

An AI-Orchestrated Travel Planning Agent

Senior Capstone Project — Final Report

Powered by 6 Specialized MCP Microservices

Project Title	SmartTravel MCP — AI-Orchestrated Travel Planning
Made By:	Abdullah Sheikh, Nasser Al-Aswad, Faizan Nadeem, Diego Romero, Azeem Indawala
Tech Stack	Python · FastMCP · SerpAPI · NWS API · GeoPy · Nominatim

1. EXECUTIVE SUMMARY

SmartTravel MCP is a travel planning agent based on the Model Context Protocol (MCP) framework that utilises the power of AI. It coordinates 6 independent microservice servers (Flight, Hotel, Weather, Geocoder, Event, Finance) to produce full end-to-end trip itineraries from a single natural language prompt. "Plan a 5 day trip to Banff for two on a \$3,000 budget" can be entered in the user interface and provide a weather-aware, culturally enriched, budget-normalized itinerary without opening a browser tab. We successfully delivered the project in 15 weeks including competitive research, microservice architecture design, core infrastructure development, advanced filtering logic, full LLM orchestration wiring and validation of a live showcase.

2. INTRODUCTION & PROBLEM STATEMENT

The irony of planning travel in the digital age is that there's too much data, yet no one tool can pull together flights, hotels, weather, local events and currency conversion into a cohesive, actionable plan. You have to flip back and forth between browser tabs, cross-check data on different platforms, and hope that Mother Nature has your back when it comes to outdoor plans. In this project we refer to this fragmentation as the API Silo Problem.

Traditional Approach (HIGH EFFORT)	SmartTravel MCP (HIGH PRECISION)
User → Tab 1 → Tab 2 → Tab 3 → Manual Documentation	User → LLM Orchestrator → 6 MCP Servers → Auto Plan

SmartTravel MCP solves this by decomposing travel planning into six specialised MCP microservices, each responsible for one domain of travel intelligence, all orchestrated by a central LLM reasoning layer. This design enables what no single API integration can: deeply contextual, real-time, budget-aware trip planning at scale.

3. SYSTEM ARCHITECTURE OVERVIEW

The system is built around a Generalist-Specialist approach. The central LLM Orchestrator acts as the leader/manager, it assigns tasks and ensures communication between agents. The 6 MCP servers are experts in their field. Using the instructions from the Orchestrator the Servers use their specialized tools to craft the best outputs possible for the user.

Architecture Principles: This framework adheres to four critical guidelines to ensure efficient collaboration between the Lead and the Experts:

- **Separation of Concerns:** Each server owns exactly one travel domain with no capability overlap. This ensures clarity once information is compiled to the user at the end.
- **Dependency Ordering:** Some tasks are important to do before others. For example the Geocoder Server must resolve coordinates before Weather or Hotel servers can execute.
- **Stateless Servers + Stateful Cache:** Server outputs are persisted as local JSON files, enabling coherent multi-turn conversations.
- **Protocol Uniformity:** All six servers implement a standardized MCP format, allowing the orchestrator the ability to query, task, and understand the agents without the need for unique modifiers.

Orchestration Flow

User Request → LLM Orchestrator → Geocoder server → Flight server → (Hotel + Weather server work in parallel)
→ Event server → Finance server → Complete Itinerary.

The orchestrator uses geocoded coordinates as a "spatial key" passed to all location-dependent servers.

Multi-server calls are coordinated with explicit dependency ordering to prevent null-input failures.

4. MCP SERVER DEEP DIVES

The following sections provide technical detail on each of the six MCP servers.

Server	Role	Backend	Description
✈️ Flight	Transit Boundary Management	SerpAPI · Google Flights	Route search, real-time pricing, schedule fetching. Local JSON caching enables multi-turn comparison without redundant API calls.
🏨 Hotel	Granular Property Discovery	SerpAPI · Google Hotels	Advanced filtering by star rating, price, and amenities. Custom NL parsing converts unstructured requests into structured JSON queries.
☀️ Weather	Atmospheric Safety & Intelligence	NWS API	Live grid-level forecasts, current conditions, and severe alert detection. Proactively triggers itinerary adjustments on hazard alerts.
📍 Geocoder	Spatial Coordinate Resolution	GeoPy · Nominatim (OSM)	Converts natural language place names into WGS84 lat/long. Foundational 'spatial key' required by Weather, Hotel, and Flight servers.
📅 Event	Cultural & Local Discovery	SerpAPI · Google Events	Discovers festivals, concerts, and attractions within the travel window, transforming logistical itineraries into cultural experiences.
💰 Finance	Real-Time Budgetary Oversight	SerpAPI · Google Finance	Live currency conversion normalises all international costs to the user's home currency before the final itinerary is assembled.

4.1 Geocoder Server — Spatial Coordinate Resolution

The Geocoder Expert is the system's foundational specialist, providing the mandatory "where" for every other location-dependent member of the squad. This expert translates common text into precise mathematical coordinates (WGS84) by utilizing a robust toolset built on GeoPy and OpenStreetMap's Nominatim service, always enforcing a strict one-second minimum delay between requests to comply with the Nominatim usage policy. The expert maintains a comprehensive, localized repository of previous work (geocoded_locations/) for immediate reference on subsequent inquiries, ensuring high efficiency.

This specialist's core capabilities include the Geocode Location tool, which converts specific place names, addresses, or landmarks into exact coordinates; the Batch Geocode tool, which is critical for processing multiple location queries simultaneously for multi-city itineraries; and the Calculate Distance tool, which computes great-circle distances (in km, miles, or nautical miles) between any two established coordinate pairs.

4.2 Weather Server — Atmospheric Safety & Intelligence

The Weather Expert provides critical atmospheric intelligence by integrating detailed, grid-level meteorological data from advanced forecasting services (the National Weather Service [NWS] API). This specialist's most vital operational role is Hazard Detection: the central Project Lead (LLM Orchestrator) utilizes the Weather Expert's real-time safety alerts to automatically recognize conflicts and immediately trigger the rescheduling of any affected outdoor activities.

To support these operations, this expert possesses a specialized toolkit, including the Get Weather Forecast tool, which delivers daily and detailed hourly NWS forecasts that enable hour-by-hour itinerary optimization; the Get Weather Alerts tool, which monitors and queries active warnings based on geography, severity, and urgency; and the Filter Forecast by Conditions tool, which is capable of analyzing and filtering forecast data based on temperature range, precipitation probability, and wind speed thresholds.

4.3 Hotel Server — Granular Property Discovery

The Hotel Server is used to search for hotels based on what the user is looking for. It does this by connecting to SerpAPI's Google Hotels API and helps return hotel options based on location, dates, guests, price, rating, and other preferences. Hotels are usually one of the main things that people compare when planning a trip. One of the more useful parts of this server is that it can understand normal user requests instead of only strict filter inputs. For example, if someone types something like "find me a pet-friendly hotel under \$150 with a pool," the system needs to break that request down into information the API can use. To do this, the server looks for keywords, similar words, and price limits in the sentence. This makes the hotel search feel a lot more natural because the user does not have to know the exact format or filters the API expects.

The search_hotels tool handles the main hotel search using details like the city, travel dates, number of guests, currency, sorting preference, and hotel class. The filter_hotels_by_price, rating, and amenities tools help narrow down the results after the first search. The get_property_details tool gives more information about a specific hotel, such as extra pricing details and booking options when they are available.

4.4 Event Server — Cultural & Local Discovery

The Event Server helps make the trip a lot more engaging and interesting by finding things and amenities to do near the destination. While the flight and hotel servers mainly focus on the basic travel needs, the Event Server adds activities and local experiences to the itinerary. This server connects to SerpAPI's Google Events API and searches for events like concerts, festivals, and also local attractions. It can also filter events by date, type, or venue, which helps make sure the events actually fit the user's trip. For example, if someone is only visiting a city for a weekend, the system should only show events happening during that weekend instead of random events outside the travel dates.

The search_events tool looks for events based on the user's request, location, date range, event type, language, and country code. The filter_events_by_date, type, and venue tools help clean up the results so the final itinerary only includes events that make sense for the user's schedule and interests.

4.5 Finance Server — Real-Time Budgetary Oversight

The Finance Server is mainly used to help with currency conversion and the budgeting. Travel costs can come from a lot of different places and also in different currencies, especially for international trips. Without this server, the final itinerary could be extremely confusing because prices might not all be shown in the same currency. This server uses SerpAPI's Google Finance API to convert prices into the user's home currency. For example, if a hotel is listed in euros but the user wants to understand the cost in U.S. dollars, the Finance Server can convert it before the final plan is shown.

The `convert_currency` tool converts one currency to another, such as USD to EUR, USD to JPY, or USD to CAD. The `get_historical_data` tool can look at past exchange rate data which is very helpful when the user wants more context on how a currency has changed over time.

4.6 Flight Server — Transit Boundary Management

The Flight Server handles the flight parts of the travel plan. It connects to SerpAPI's Google Flights API and is used to search for routes, prices, and schedules. Since flights usually decide when the trip starts and ends, this server helps set the basic structure as well as the foundation for the rest of the itinerary. A major feature of the Flight Server is that it saves flight results in local JSON files. Each search gets stored with a unique search ID, so that the system can use those results again later. This matters for follow-up questions. For example, if a user asks for flights and then says "show me something cheaper," the system can look back at the saved results instead of starting the whole search over again. It makes it feel like a real conversation because the system remembers the results from the previous step. The Flight Server can also filter flights by price, number of stops, and flight duration, which makes it easier for the user to compare different options. The Flight Server gives the trip its travel schedule and helps the orchestrator build the rest of the plan around realistic flight options.

5. AGENT ORCHESTRATION & METHODOLOGY

The LLM Orchestrator receives raw user input, determines which servers to invoke and in what order, passes outputs as context between servers, and synthesises the final itinerary. All six MCP servers offer their services as callable utilities through the default MCP API interface.

5.1 Dependency-aware Multi-Server Routing

The orchestrator applies a dependency-aware routing algorithm to execute requests. Geographic entities are detected first, and then if coordinate resolution is needed, the Geocoder Server is invoked with its output injected into the context of all other servers. Both Hotel and Weather servers can be invoked concurrently, as they rely only on geocoded coordinates. The Event Server request comes next using the travel window defined by the Flight Server. Finally, the Finance Server calculates overall costs.

5.2 JSON State Caching for Multi-Turn Persistence

LLMs lack any kind of persistent memory across API calls. To avoid fetching data anew in every follow-up request, the SmartTravel MCP maintains a local JSON cache of each server output with a unique search ID. In case of multi-turn conversations initiated by the user, such as switching hotels or checking other dates, the orchestrator loads the corresponding JSONs from cache, saving on latencies and API calls and facilitating context-sensitive dialogue.

5.3 Natural Language to Structured Query Translation

The natural language parser of the Hotel Server plays an important technical role because it translates the natural descriptions of hotel amenities into their corresponding filter identifiers in the SerpAPI filter system by means of a synonym resolution process and also converts prices in free text form into regular expressions using the normalisation process.

6. DEVELOPMENT TIMELINE

Timeline	Phase	Key Deliverables
Wks 1–3	Phase 1: Research	After examining the available travel applications, we realized

		the recurring theme of everything being siloed – no application has been created to consolidate information related to flights, hotels, weather, and events. This is what formed our problem statement. We eventually zeroed in on using MCP, SerpAPI, NWS API, and GeoPy based on comparisons of other options.
Wks 4–5	Phase 2: Architecture	Determined how the six servers would communicate and determined the order of dependencies. The key choice in this regard was selecting the Geocoder server as the base from which all other operations would flow.
Wks 6–8	Phase 3: Core Infrastructure	MCP environments were already set up, and then I made the Geocoder and Weather services, which were the foundation for all others to come. During this phase, the Flight server was also created, along with the JSON caching layer that proved helpful for multi-turn interactions.
Wks 9–11	Phase 4: Refinement	This was the toughest period. The processing time for the Hotel Server using the natural language parser was far longer than anticipated, and ensuring that the parser could accurately process input such as "pet friendly below \$150 with a swimming pool" involved a significant amount of trial and error.
Wks 12–13	Phase 5: Integration	LLM orchestration layer is where we integrated and were able to get the six servers communicating with one another end-to-end. There were many bugs revealed here that we were not able to detect via unit testing, mainly due to timing and server-to-server context passing.
Wks 14–15	Phase 6: Senior Design Day	We were able to resolve some of the problems we faced with API rate limits. Then, we completed the architecture diagrams and

		the poster and prepared ourselves for the presentation.
--	--	---

7. KEY ENGINEERING CHALLENGES

- **API Rate Limiting & Schema Mismatches:** All six SerpAPI integrations required defensive parsing and null-safe accessors to handle inconsistent response schemas and missing fields. The NWS API required specific User-Agent headers and has coverage limited to the United States. Exponential backoff retry logic was implemented across all six servers to handle transient rate-limit responses.
- **Server Dependency Ordering:** Early integration testing revealed that the orchestrator could attempt parallel server calls before geocoding had completed, resulting in null coordinate inputs. This was resolved with explicit dependency declarations in the orchestration layer, enforcing Geocoder-first sequencing for all location-bearing queries.
- **Stateless LLM Memory:** Designing a unique-ID-based JSON caching schema that the orchestrator could reliably reason about required careful prompt engineering alongside the storage implementation. The cache key structure, file naming convention, and retrieval prompts were iterated several times before achieving stable multi-turn performance.
- **Natural Language Filtering Complexity:** Requests like "pet-friendly hotel under \$150 with free parking" required a synonym resolution layer (mapping terms such as 'gym', 'fitness centre', and 'exercise room' to the same SerpAPI filter ID) and regex-based price extraction. Edge cases — price ranges, minimum/maximum bounds, compound amenity lists — drove the majority of this component's development time.
- **Multi-Server Integration Testing:** Isolated unit tests were insufficient to expose inter-server data-flow failures. Only live end-to-end test runs — exercising the full orchestration pipeline with real API keys — revealed the timing, schema, and context-passing issues that could not be caught in isolation.

8. CONCLUSION & FUTURE WORK

SmartTravel MCP shows how orchestrated AI microservice architectures can fundamentally change travel planning. The system decomposes the problem into six specialized MCP servers wired through a central LLM orchestrator, and provides capabilities that no single travel API can provide: real-time, weather-aware, culturally enriched, and budget-normalized itinerary generation from a single natural language prompt.

Project Accomplishments

- **Architecture:** Implemented and deployed a fully functional MCP microservice system with 6 servers and multi-server LLM orchestration routing.
- **Performance:** End-to-end trip planning from a single natural language prompt, replacing 3+ browser tabs with a single automated pipeline.
- **Validation:** System architecture validated through live API testing, multi-turn conversational flows and complex multi-destination itinerary generation.
- **Engineering:** Built non-trivial technical components: NL-to-query parsing, dependency-ordered orchestration and Json multi-turn state caching.

Limitations

NWS weather coverage is only within the United States, international trips would need to be integrated with Open-Meteo or OpenWeatherMap. Flight data is for search and comparison only, no booking actions. SerpAPI's free tier (100 searches/month) doesn't scale for production use without a paid subscription.

Future Directions

- Global Weather Add Open-Meteo global weather
- Direct Booking: Add booking actions through Amadeus or Booking.com Partner API to close the loop from planning to reservation.
- User Profiles: Develop persistent preference profiles for dietary needs, accessibility and loyalty programme memberships.
- Expanded MCP Ecosystem: Add car rental, travel insurance, visa requirement, and real-time transit servers for trip coverage.

SmartTravel MCP completes its 15-week cycle as a validated, extensible foundation for next-generation AI-powered travel intelligence, where an entire journey from inspiration to itinerary is orchestrated through a single conversation.